



VANLANG UNIVERSITY
School of Technology
Information Technology



HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

(Database Management System)

CHƯƠNG 3: STORED PROCEDURE & TRIGGER



GVGD: THS. NGUYỄN THỊ QUYÊN



HỌC KỲ II – NĂM HỌC 2024-2025



KHÓA 29T-IT





NỘI DUNG

- 01.** CÁC LOẠI LẬP TRÌNH THỦ TỤC
- 02.** GIỚI THIỆU STORED PROCEDURE
- 03.** CÚ PHÁP STORED PROCEDURE
- 04.** ĐẶT VẤN ĐỀ TRIGGER
- 05.** GIỚI THIỆU TRIGGER
- 06.** CÚ PHÁP TRIGGER

1. CÁC LOẠI LẬP TRÌNH THỦ TỤC

Loại	Nhóm câu lệnh	Lưu trữ	Thực thi	Tham số
Script	Nhiều nhóm câu lệnh	File trên ổ đĩa	Từ công cụ Client như Management Studio	Không
Stored Procedure	Duy nhất	Đối tượng của CSDL	Bởi ứng dụng hoặc trong mã kịch bản SQL	Có
Function	Duy nhất	Đối tượng của CSDL	Bởi ứng dụng hoặc trong mã kịch bản SQL	Có
Trigger	Duy nhất	Đối tượng của CSDL	Tự động bởi Server khi có 1 action xảy ra	Không



2. GIỚI THIỆU STORED PROCEDURE

2.1 STORED PROCEDURE (SP) LÀ GÌ?

- **SP là một tập các câu lệnh thực hiện một nhiệm vụ cụ thể, được đặt tên và lưu trữ trong CSDL dạng đã biên dịch**
- **SP cung cấp một giải pháp hữu ích cho việc thực thi lặp lại cùng một nhiệm vụ**
 - Giúp tái sử dụng code
 - Khi thực thi lại một nhiệm vụ, sử dụng lời gọi SP thay vì viết và thực thi lại cùng một tập hợp các câu lệnh



2. GIỚI THIỆU STORED PROCEDURE

2.2 KHẢ NĂNG CỦA STORED PROCEDURE (SP)

- Các cấu trúc điều khiển (IF, WHILE, FOR) có thể được sử dụng trong thủ tục.
- Bên trong thủ tục lưu trữ có thể sử dụng các biến như trong ngôn ngữ lập trình nhằm lưu giữ các giá trị tính toán được, các giá trị được truy xuất được từ cơ sở dữ liệu.
- Một tập các câu lệnh SQL được kết hợp lại với nhau thành một khối lệnh bên trong một thủ tục.



2. GIỚI THIỆU STORED PROCEDURE

2.3 ĐẶC TRƯNG CỦA STORED PROCEDURE (SP)

- Tên thủ tục nội tại
- Tham số truyền giá trị vào
- Tham số nhận giá trị trả ra
- Được gọi thực hiện trong môi trường không phải là Microsoft SQL Server.
- Thực thi khá nhanh.

3. CÚ PHÁP STORED PROCEDURE

3.1 CÚ PHÁP

```
CREATE PROC[EDURE] Tên_thủ_tục  
[@Biến_toàn_cục datatype,  
...]  
AS  
[DECLARE Biến_cục_bộ]  
  
Các_lệnh
```

```
void tinhTong2(int a, int b) {  
    int tong;  
    tong = a + b;  
    printf("Tong hai so la %d", tong);  
}
```

- **Tên thủ tục:** tên thủ tục nội tại được tạo mới là **duy nhất**.
- **Biến toàn cục:** là những biến được sử dụng trong phạm vi toàn bài toán, người dùng không cần khai báo.
- **Biến cục bộ:** là những biến cục bộ được sử dụng tính toán tạm thời bên trong thủ tục, những biến này chỉ có phạm vi cục bộ bên trong thủ tục nội tại.
- **Các lệnh:** các lệnh bên trong thủ tục nội tại dùng để xử lý tính toán theo một yêu cầu nào đó.

3. CÚ PHÁP STORED PROCEDURE

3.2 VÍ DỤ: THỦ TỤC KHÔNG THAM SỐ

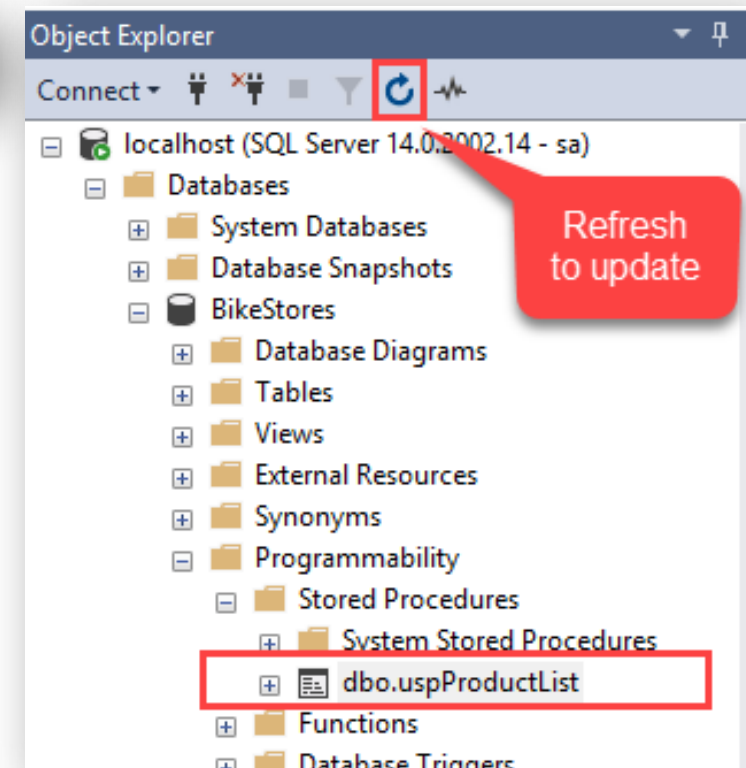
Thủ tục liệt kê danh sách

```
1 CREATE PROCEDURE uspProductList
2 AS
3 BEGIN
4     SELECT
5         product_name,
6         list_price
7     FROM
8         production.products
9     ORDER BY
10        product_name;
11 END;
```

Thực thi thủ tục

```
1 EXEC uspProductList;
```

Thủ tục lưu trữ



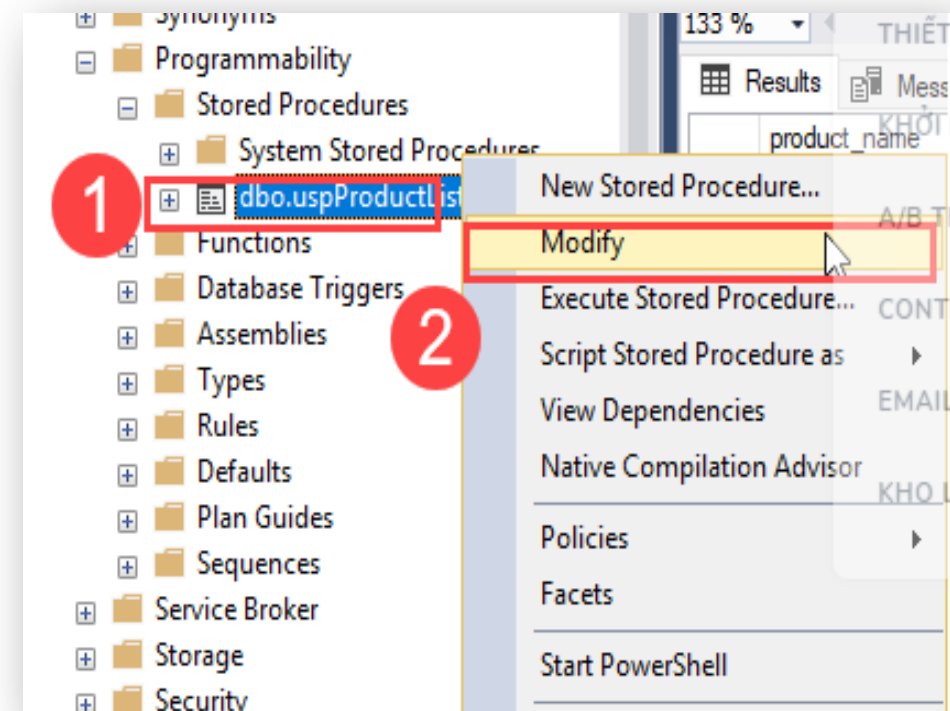
3. CÚ PHÁP STORED PROCEDURE

3.3 VÍ DỤ: THAY ĐỔI THỦ TỤC

Thay đổi thủ tục đã có:

```
1 ALTER PROCEDURE uspProductList
2 AS
3 BEGIN
4     SELECT
5         product_name,
6         list_price
7     FROM
8         production.products
9     ORDER BY
10        list_price
11 END;
```

Có thể thực hiện bằng giao diện:





3. CÚ PHÁP STORED PROCEDURE

3.4 VÍ DỤ: XÓA THỦ TỤC

Cú pháp:

```
1 | DROP PROCEDURE sp_name;
```

Hoặc là

```
1 | DROP PROC sp_name;
```

Ví dụ xóa thủ tục vừa tạo

```
1 | DROP PROCEDURE uspProductList;
```

3. CÚ PHÁP STORED PROCEDURE

3.5 VÍ DỤ: THỦ TỤC CÓ BIẾN (THAM SỐ) TOÀN CỤC

- Truyền tham số để xuất những sản phẩm có giá bán $\geq$ giá gốc:

```
1 ALTER PROCEDURE uspFindProducts
2 (
3     @min_list_price AS DECIMAL
4 )
5 AS
6 BEGIN
7     SELECT
8         product_name,
9         list_price
10    FROM
11        production.products
12   WHERE
13       list_price >= @min_list_price
14   ORDER BY
15       list_price;
16 END;
```

- Biến toàn cục: bắt đầu bằng chữ @
- Cú pháp:
<<tên biến>> <<kiểu dữ liệu>> [<<=giá trị mặc định>>] [OUTPUT]
- Lưu ý: khi có tham số thì nên sử dụng trong thân SP.
- Thực thi thủ tục có tham số:

```
1 EXEC uspFindProducts 100;
```

3. CÚ PHÁP STORED PROCEDURE

3.6 VÍ DỤ: THỦ TỤC CÓ BIẾN (THAM SỐ) TOÀN CỤC TÙY CHỌN

- Những tham số mặc định có thể truyền hoặc không:

```
1 ALTER PROCEDURE uspFindProducts
2 (
3     @min_list_price AS DECIMAL = 0,
4     @max_list_price AS DECIMAL = 999999,
5     @name AS VARCHAR(max)
6 )
7 AS
8 BEGIN
9     SELECT
10         product_name,
11         list_price
12     FROM
13         production.products
14     WHERE
15         list_price >= @min_list_price AND
16         list_price <= @max_list_price AND
17         product_name LIKE '%' + @name + '%'
18     ORDER BY
19         list_price;
20 END;
```

- Thực thi thủ tục có tham số tùy chọn:

```
1 EXEC uspFindProducts
2     @name = 'Trek';
```

3. CÚ PHÁP STORED PROCEDURE

3.7 VÍ DỤ: THỦ TỤC CÓ BIẾN (THAM SỐ) CỤC BỘ

- Xuất danh sách sản phẩm cộng

```
1 CREATE PROC uspGetProductList
2 (
3     @model_year SMALLINT
4 ) AS
5 BEGIN
6     DECLARE @product_list VARCHAR(MAX);
7
8     SET @product_list = '';
9
10    SELECT
11        @product_list = @product_list + product_name
12                        + CHAR(10)
13    FROM
14        production.products
15    WHERE
16        model_year = @model_year
17    ORDER BY
18        product_name;
19
20    PRINT @product_list;
21 END;
```

- Thực thi thủ tục:

```
1 EXEC uspGetProductList 2018
```

3. CÚ PHÁP STORED PROCEDURE

3.8 VÍ DỤ: THỦ TỤC CÓ BIẾN (THAM SỐ) TOÀN CỤC OUTPUT

- Xuất sản phẩm theo năm model và trả về SL sản phẩm thông qua biến OUTPUT:

```
1 CREATE PROCEDURE uspFindProductByModel
2 (
3     @model_year SMALLINT,
4     @product_count INT OUTPUT
5 ) AS
6 BEGIN
7     SELECT
8         product_name,
9         list_price
10    FROM
11        production.products
12   WHERE
13        model_year = @model_year;
14
15    SELECT @product_count = @@ROWCOUNT;
16 END;
```

- Thực thi thủ tục:

```
1 DECLARE @count INT;
2
3 EXEC uspFindProductByModel
4     @model_year = 2018,
5     @product_count = @count OUTPUT;
6
7 SELECT @count AS 'Number of products found';
```

3. CÚ PHÁP STORED PROCEDURE

3.9 VÍ DỤ: TRY... CATCH TRONG STORED PROCEDURE

• Bảng A:

	A1	A2
1	1	1
2	2	2
3	3	3
4	4	4

• Bảng B:

	B1	B2	A_B
1	1	1	1
2	2	2	1
3	3	3	2
4	4	4	NULL

• Stored Procedure ban đầu:

```
CREATE PROCEDURE deleteA
    @A1 int
AS
BEGIN
    DELETE FROM A WHERE A1 = @A1
END
GO
```

```
EXEC deleteA @A1=1
GO
```

Lệnh này sẽ phát sinh lỗi do B(1), B(2) đang tham chiếu đến A(1)

• Sửa lại dùng Try...Catch:

```
ALTER PROCEDURE deleteA
    @A1 int
```

```
AS
```

```
BEGIN TRY
```

```
    DELETE FROM A WHERE A1 = @A1
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'ERROR on delete record: ' + CONVERT(varchar, @A1)
```

```
    RETURN 1001 -- return user-defined error code
```

```
END CATCH
```

```
GO
```

3. CÚ PHÁP STORED PROCEDURE

3.10 BÁO LỖI: RAISERROR (1/2)

- Cú pháp

```
1 RAISERROR ( { msg_id | msg_str | @local_variable }  
2           { ,severity ,state }  
3           [ ,argument [ ,...n ] ] )  
4           [ WITH option [ ,...n ] ]
```

- Tham số đầu tiên: nội dung thông báo lỗi.
 - msg_id: Là một 'error message' được người dùng khai báo. Nó được lưu trữ trong sys.message.
 - msg_str: Là một chuỗi lỗi, được đặt trong dấu nháy đơn.
 - @local_variable: Một biến cục bộ, chứa thông báo lỗi.

Tham số thứ hai: mức độ của lỗi

- severity:
 - 0-10: Informational messages
 - 11-18: Errors
 - 19-25: Fatal errors
- state: là một số nằm trong đoạn [0-255]. Mỗi giá trị sẽ mang một ý nghĩa được quy ước bởi quản trị viên, có thể set 1 giá trị chung, để có phương án xử lý.



3. CÚ PHÁP STORED PROCEDURE

3.10 BÁO LỖI: RAISERROR (2/2)

- Ví dụ:

```
BEGIN TRY
    RAISERROR(N'Test', 16, 1);
    SELECT 1;    /* Not executed */
END TRY
BEGIN CATCH
    SELECT 2;    /* Executed */
END CATCH
```

```
BEGIN TRY
    RAISERROR(N'Test', 10, 1);
    SELECT 1;    /* Executed */
END TRY
BEGIN CATCH
    SELECT 2;    /* Not executed */
END CATCH
```

4. ĐẶT VẤN ĐỀ

TAIKHOAN

MaTK	TenTK	SoDuTK
16111	Mi Nguyen	1.000.000
16222	Chau Ly	2.000.000

GIAODICHRUTIENTIEN

MaGD	MaTK	ViTriGD	NgayGD	SoTienGD
001	16111	QGD	1/1/2021	1.000.000
002	16222	ATM	2/3/2021	2.000.000
003	16222	ATM	6/9/2021	3.000.000

- Quy tắc nghiệp vụ:

- Số tiền rút tối thiểu: **50.000**
- Số tiền rút tối đa:
 - Quầy giao dịch: số tiền không nhiều hơn số dư
 - ATM: số tiền không nhiều hơn số dư hoặc không nhiều hơn **3.000.000**
- Số dư tài khoản sau rút = $\text{SoDuTK} - \text{SoTienGD}$



5. GIỚI THIỆU TRIGGER

5.1 TRIGGER LÀ GÌ?

- Trigger là một loại stored procedure đặc biệt được thực thi (execute) một cách tự động khi có một sự kiện thay đổi dữ liệu (data modification) xảy ra như **Update**, **Insert** hoặc **Delete**.
- Trigger được dùng để đảm bảo tính toàn vẹn dữ liệu (Data Integrity) hoặc thực hiện các quy tắc nghiệp vụ (business rules) nào đó.



5. GIỚI THIỆU TRIGGER

5.2 MỘT SỐ XỬ LÝ

- Kiểm tra ràng buộc dữ liệu
- Những tính toán nghiệp vụ cần thiết có liên quan
- Lưu vết các hoạt động



5. GIỚI THIỆU TRIGGER

5.3 BIẾN CỐ KÍCH HOẠT TRIGGER

- INSERT
- UPDATE
- DELETE



5. GIỚI THIỆU TRIGGER

5.4 ĐẶC ĐIỂM CỦA TRIGGER

- Một trigger có thể làm nhiều công việc (actions) khác nhau và có thể được kích hoạt bởi nhiều hơn một event.
- Trigger không thể được tạo ra trên bảng tạm.
- Trigger chỉ có thể được kích hoạt một cách tự động bởi một trong các event **Insert**, **Update**, **Delete** mà không thể chạy manually được.
- Có thể áp dụng trigger cho View.



5. GIỚI THIỆU TRIGGER

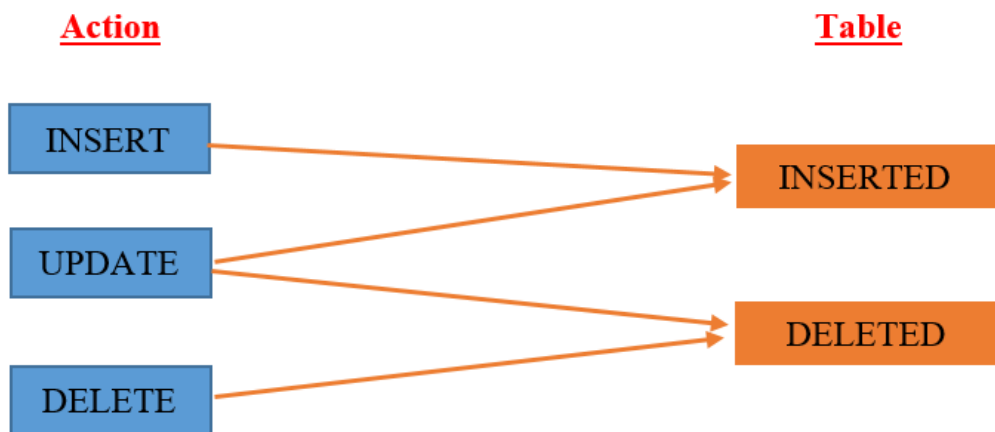
5.5 BẢNG LOGIC (1/2)

- Khi một trigger được kích hoạt thì data mới vừa được insert hay mới vừa được thay đổi sẽ được chứa trong bảng **Inserted** còn data mới vừa được delete được chứa trong bảng **Deleted**.
- Dữ liệu 2 table tạm Inserted và Deleted: chỉ chứa trên memory và chỉ có giá trị bên trong trigger.

Vậy khi Update (cập nhật dữ liệu) thì dữ liệu được chứa trong bảng nào?

5. GIỚI THIỆU TRIGGER

5.5 BẢNG LOGIC (2/2)



- Hoặc khi muốn cập nhật Mặt hàng '**H8**' (action Update) như sau:

UPDATE MATHANG

SET SOLUONG=40, TENHANG = N'Nước xả'

WHERE MAHANG = 'H8'

Thì 2 Bảng **Inserted** và **Deleted** chứa dữ liệu như sau:

Deleted Table

MAHANG	TENHANG	SOLUONG
H8	Nước tẩy	30

Inserted Table

MAHANG	TENHANG	SOLUONG
H8	Nước xả	40

- Khi thêm mới một MATHANG (action INSERT) như sau:

INSERT INTO MATHANG VALUES ('H8',N'Nước tẩy',30)

Thì Bảng Inserted chứa dữ liệu như sau:

MAHANG	TENHANG	SOLUONG
H8	Nước tẩy	30

- Khi xóa MATHANG '**H8**' (action Delete) như sau:

DELETE MATHANG WHERE MAHANG = 'H8'

Thì Bảng Deleted chứa dữ liệu như sau:

MAHANG	TENHANG	SOLUONG
H8	Nước tẩy	30



6. CÚ PHÁP TRIGGER

6.1 CÚ PHÁP CÀI ĐẶT (1/3)

```
CREATE TRIGGER tên_trigger  
ON tên_bảng  
{[INSTEAD OF] | [FOR | AFTER ] } {[INSERT][,][UPDATE][,][DELETE]}  
AS  
  
[DECLARE Biến_cục_bộ]  
  
[IF UPDATE(tên_cột)  
[AND UPDATE(tên_cột)|OR UPDATE(tên_cột)  
...]  
  
Các_câu_lệnh_của_trigger
```

6. CÚ PHÁP TRIGGER

6.1 CÚ PHÁP CÀI ĐẶT (2/3)

```
CREATE TRIGGER tên_trigger
ON tên_bảng
{[INSTEAD OF] | [FOR | AFTER] } {[INSERT][,][UPDATE][,][DELETE]}
AS
[DECLARE Biến_cục_bộ]

[IF UPDATE(tên_cột)
[AND UPDATE(tên_cột)|OR UPDATE(tên_cột)
...]

Các_câu_lệnh_của_trigger
```

- **Tên trigger:** tên trigger được tạo mới, tên trigger này phải là **duy nhất** trong một cơ sở dữ liệu
- **Tên bảng:** tên bảng có trong cơ sở dữ liệu mà trigger tạo mới có liên quan đến.
- **INSTEAD OF:** Trigger thực hiện khi **chưa** thực hiện Action. Chú ý rằng với mỗi bảng, bạn chỉ có quyền tạo **một instead of trigger** cho một hành động cập nhật dữ liệu.
- **FOR hoặc AFTER:** Trigger thực hiện khi **đã** thực hiện Action. Nếu tạo trigger thông thường hay dùng từ khoá **For**.



6. CÚ PHÁP TRIGGER

6.1 CÚ PHÁP CÀI ĐẶT (3/3)

```
CREATE TRIGGER tên_trigger
ON tên_bảng
{[INSTEAD OF] | [FOR | AFTER] } {[INSERT][,][UPDATE][,][DELETE]}
AS
[DECLARE Biến_cục_bộ]

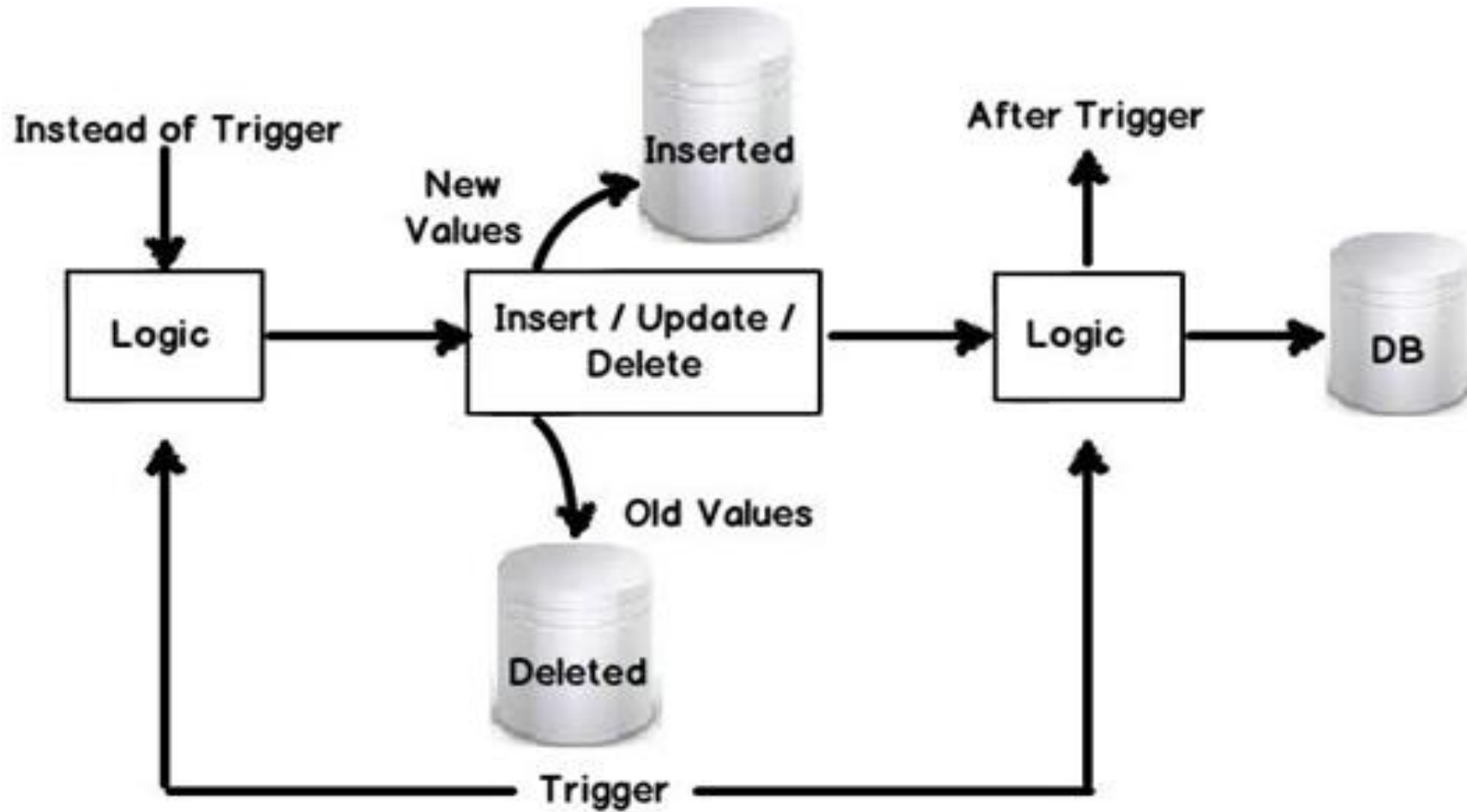
[IF UPDATE(tên_cột)
[AND UPDATE(tên_cột)|OR UPDATE(tên_cột)
...]

Các_câu_lệnh_của_trigger
```

- **INSERT, UPDATE, DELETE:** các hành động cập nhật dữ liệu có liên quan tác động vào bảng để kích hoạt trigger.
- **IF UPDATE:** Trigger chỉ thực thi khi hành động cập nhật được thực hiện trên cột được liệt kê trong IF UPDATE.
- **Biến cục bộ:** là những biến cục bộ được sử dụng trong trigger
- **Các câu lệnh:** các lệnh bên trong trigger dùng để kiểm tra các ràng buộc toàn vẹn dữ liệu.

6. CÚ PHÁP TRIGGER

6.2 CƠ CHẾ HOẠT ĐỘNG





6. CÚ PHÁP TRIGGER

6.3 INSTEAD OF & FOR/AFTER

- **INSTEAD OF:**

- Trigger được gọi thực hiện thay cho thao tác Insert/Update/Delete.
- Các dòng mới chỉ được thêm Inserted (bảng thật chưa thêm)
- Các dòng bị xóa nằm trong bảng Deleted (bảng thật chưa xóa)

- **FOR/AFTER:**

- Trigger được gọi thực hiện sau khi thao tác Insert/Update/Delete được thực hiện.
- Các dòng mới được thêm vào **Inserted** và bảng thật.
- Các dòng bị xóa nằm trong bảng **Deleted** và bảng thật bị mất dữ liệu bị xóa
- Có thể quay lui thao tác đã thực hiện bằng Rollback Transaction.

6. CÚ PHÁP TRIGGER

6.4 VÍ DỤ:

TAIKHOAN

MaTK	TenTK	SoDuTK
16111	Mi Nguyen	1.000.000
16222	Chau Ly	2.000.000

INSERTED

MaGD	MaTK	ViTriGD	NgayGD	SoTienGD
003	16222	ATM	6/9/2021	1.000.000

GIAODICHRUTTIEN

MaGD	MaTK	ViTriGD	NgayGD	SoTienGD
001	16111	QGD	1/1/2021	1.000.000
002	16222	ATM	2/3/2021	5.000.000
003	16222	ATM	6/9/2021	1.000.000

```
create trigger trg_ThemGDRutTien
on GiaoDichRutTien
after INSERT
```

```
as
```

```
declare @vitri varchar(10), @matk char(10)
```

```
declare @tienrut int
```

```
declare @sodu int
```

```
select @vitri=ViTriGD, @tienrut=SoTienGD, @sodu=SoDuTK
from inserted inner join TaiKhoan
on inserted.MaTK = TaiKhoan.MaTK
```

```
if @tienrut>@sodu
```

```
begin
```

```
raiserror(N'Không được rút nhiều hơn số dư',16,1)
```

```
rollback transaction
```

```
return
```

```
end
```

```
if @vitri='ATM' and @tienrut>3000000
```

```
begin
```

```
raiserror(N'Không rút nhiều hơn 3trieu', 16,1)
```

```
rollback transaction
```

```
end
```

6. CÚ PHÁP TRIGGER

6.5 IF UPDATE:

- Khi muốn trigger thực sự hoạt động khi một hay vài column nào đó được Update chứ không phải bất kỳ column nào.
- Khi đó ta có thể dùng hàm **Update** (*Column_Name*) để kiểm tra xem column nào đó có bị update hay không.
- IF UPDATE không sử dụng được đối với câu lệnh DELETE.

```
CREATE TRIGGER reminder
ON Person.Address
AFTER UPDATE
AS
IF ( UPDATE (StateProvinceID) OR UPDATE (PostalCode) )
BEGIN
    RAISERROR (50009, 16, 10)
END;
GO

-- Test the trigger.
UPDATE Person.Address
SET PostalCode = 99999
WHERE PostalCode = '12345';
GO
```



6. CÚ PHÁP TRIGGER

6.6 CÚ PHÁP KHÁC

- Xóa Trigger
- Liệt kê tất cả các đối tượng mà trigger tham chiếu đến
- Hiển thị tất cả những Trigger có trong CSDL
- Hiển thị thông tin một Trigger
- Hiển thị nội dung một Trigger

```
Drop Trigger trigger_name
```

```
sp_depends trigger_name
```

```
SELECT * FROM sysobjects WHERE type='TR'
```

```
sp_help trigger_name
```

```
sp_helptext trigger_name
```




VANLANG UNIVERSITY
School of Technology
Information Technology

